

Lab Instructions 3.2 – Creating child processes

Target	Related theory	Resources
Create process	Ch3: Process Creation	Use Ubuntu 14/16 image
Cancel the process	Ch3: Process Creation	

Update

2021.Jun.2	Add example code in the theory book.
------------	--------------------------------------

Ask students: Understand the theory involved. Know how to compile sample code and execute it. Know how to use the fork() call to create a child process. Apply multitasking programming by creating child processes. Applying creates a process tree and the process terminates synchronously.

Student reviews: Questions and answers about theoretical issues. Practical skills. Exercise.

Submission requirements: .c source files and .out executable files for the “examples” and exercises at the end of the tutorial.

Preferences

[1] Abraham Silberschatz, Peter B. Galvin, Greg Gagne, [2018], Operating System Concepts, 10th edition, John Wiley & Sons, New Jersey.

Programming Problems of Chapter 3.

[2] Greg Gagne , [2019], GitHub OS-BOOK OSC10e, Westminster College, United States

Access <https://github.com/greggagne/osc10e> in September 2019.

[3] Linux manual page, [2021], on fork(2)

Access <https://man7.org/linux/man-pages/man2/fork.2.html>

Requirements 1: Use the fork() call in the “unistd.h” library to create a child process, the child process executes the ls call.

Instruct: Download file LAB_IntroOS/CHAPTER 3/Fig3_08_Fork.c, then compile and execute and observe.

?	- What function does the ls command have? - What is the fork() call used for? - What does a child process inherit from its parent process? - How to separate the code so that the parent and child processes execute separate blocks of code?	LAB 1 Ch3 p.151	Slide Ch3.
---	---	--------------------	------------

- Line 11 is to declare the pid variable to receive the return results when calling fork() in line 13.
- Line 14 occurs when fork() fails.
- Line 18 indicates the following block of code that will be copied to the child process.
- Line 21 shows the command of the parent process (current process).
- Line 20 is a call to execute a command (see also LAB 3.3).
- Line 24 is the parent process's wait, only when the child process completes will the parent process pass this command.

```
11    pid_t pid;
13    pid = fork();
14    if (pid < 0) {
18    } else if (pid == 0) {
20    execlp("/bin/ls", "ls", NULL);
21    } else {
24    wait(NULL);
```

The pid_t fork(void) process constructor is used with the pid_t pid=fork() call:

If successful and pid =0: is in the child process body.

If successful and pid>0: processed in the parent process body.

If failure: pid=-1 with reason, for example ENOMEM: not enough memory, or EAGAIN: number of processes exceeds the allowed limit.

Requirement 2: Complete the code so that a parent process spawns a child process, each process prints their ID to the screen, renames the process on the system, and synchronization ends between parent and child.

Instruct: Download the file LAB_IntroOS/LAB_3/task2_2.c, then compile and execute and continue to complete.

- sleep(10) in the child process will cause the child process to terminate later, while the parent process will terminate first. The child process is then called an orphan process. Please place the wait() command in the appropriate position in the parent process so that the parent waits and finishes later.

- Change the name of the process with the command. After changing, observe in System Monitor the change results.

```
#include <sys/prctl.h>
const char * name = "New_named";
if (prctl(PR_SET_NAME, (unsigned long) name) < 0) perror("prctl()");
```

Programming problem

3.1 When running the following code, what is printed to the screen by command line A (line 15)?

01	#include <sys/types.h>
02	#include <stdio.h>
03	#include <unistd.h>
04	int value = 5;
05	int main()
06	{
07	pid_t pid;
08	pid = fork();
09	if (pid == 0) { /* child process */
ten	value += 15;
11	return 0;
twe	}
lft	else if (pid > 0) { /* parent process */
h	wait(NULL);
13	printf("PARENT: value = %d",value); /* LINE A */
14	return 0;
15	}
16	}

Hint: What do students conclude about the data of the parent process and child process? Does the value variable declared in line 5 have a "global" meaning? If I want to share data between two parent and child processes, is it possible?

3.2 How many processes are created after running the program below, including the original parent process?

Hint: when processes terminate, they will be removed from "System Monitor", to make processes last longer, add the sleep(x) command before line 11 with the appropriate x parameter.

?	- What does the born child process inherit from the parent process? - Can a child process and parent process share a previously declared variable?	Ch3 p.151	Slide Ch3.
---	---	-----------	------------

01	#include <stdio.h>
02	#include <unistd.h>
03	int main()
04	{
05	/* fork a child process */
06	fork();
07	/* fork another child process */
08	fork();
09	/* and fork another */
10	fork();
11	return 0;
12	}

Programming exercises

1. Write a program to pass the positive integer n into and
 - The parent process continues to calculate and output the total $S1 = 1 + 2 + \dots + n$
 - At the same time, create a child process that calculates the sum S2 of the divisors of n and prints it to the screen.

Hints for this exercise at LAB_IntroOS/LAB_3/ex1_Pseudocode.c

2. Programming Problems 3.21 in book [1]

Collatz conjecture¹ believe that the generated number sequence will always approach 1 with any positive integer generated in the first step. The number sequence is created according to the following algorithm:

$$n = \begin{cases} \frac{n}{2} & \text{nếu } n \text{ là số chẵn} \\ 3 * n + 1 & \text{nếu } n \text{ là số lẻ} \end{cases}$$

For example, with n=35, the resulting sequence is 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1

Write a program that accepts a positive integer n as an argument and checks the correctness of this value. Create a child process to calculate and print the sequence of numbers, while the parent process needs to wait for the child process to complete through the wait() call.

```
>./collazt 35
35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1
```

Hints for this exercise at LAB_IntroOS/LAB_3/ex2_Pseudocode.c

CHECKLIST LAB

- ☐ Start Ubuntu.
- ☐ Run Terminal.
- ☐ Edit a text file .txt, .c
- ☐ Compile and Link a .c source code
- ☐ Create a statically linked library.
- ☐ Create dynamic link library.
- ☐ Run a compiled program.
- ☐ Pass arguments via command line.
- ☐ Create child process.
- ☐ Source code structure for parent process and child process.
- ☐ Create a process tree
- ☐ Synchronously terminate the parent/child process.
- ☐ Call to run a command from within the program.

¹ https://en.wikipedia.org/wiki/Ph%E1%BB%8Fng_%C4%91o%C3%A1n_Collatz